SemanticLens: Design and Implementation of an NLP-Based Retrieval Website

Andres, Ahmed, Sanabel, and Carl

Abstract

SemanticLens is an NLP-based retrieval platform intended for CLEF IAPR TC-12 annotated image retrieval. This project tackles a common problem in information retrieval: users write queries in language that does not match the exact vocabulary of a dataset, and developers need ranking methods that are transparent, testable and computationally efficient. To solve that problem, SemanticLens integrates lexical search, expressive query operators, stemming, semantic expansion, context-aware association, and TF-IDF ranking into a Next.js and TypeScript web application. This article describes the system requirements, design decisions, implementation, limitations, and future testing. It also situates the project within broader discussions of keyword matching, semantic retrieval, interface transparency and the tradeoff between lean custom systems and industry search platforms. The main argument is that SemanticLens is valuable not because it replaces large-scale search engines, but because it exposes the logic of information retrieval in a form that can be studied, tested and extended.

Keywords: information retrieval, natural language processing, semantic search, TF-IDF, Next.js, TypeScript, CLEF IAPR TC-12.

1. Introduction

SemanticLens was developed as a retrieval website to query annotated image data with natural-language queries. It is both a practical tool and an analytical study. The application connects a UI/UX with a simple search engine using the CLEF IAPR TC-12 dataset. A user can type ordinary text, specific words or phrases, Boolean expressions, wildcard characters, proximity operators, or semantically advanced words. The system returns a ranked list of image annotations together with their metadata. The project is best understood as both a software artifact and a retrieval design study. On a technical level, it shows how React, Next.js, and TypeScript can support an interactive search system backed by a server-side index. This raises an old question in information retrieval: should a search system only match exact terms, or should it give greater weight to meaning? SemanticLens does not pretend to be settling that debate. It builds around it. Lexical evidence, semantic expansion, user control and the signals of ranking all function in concert. In this paper, each software feature is treated as a response to a retrieval problem, not as an end in itself. The design is guided by vocabulary mismatch, ranking transparency, missing image files, interface complexity and maintainability. That framing matters. A search website is not just an interface: it embodies a theory of how information is represented, matched and explained.

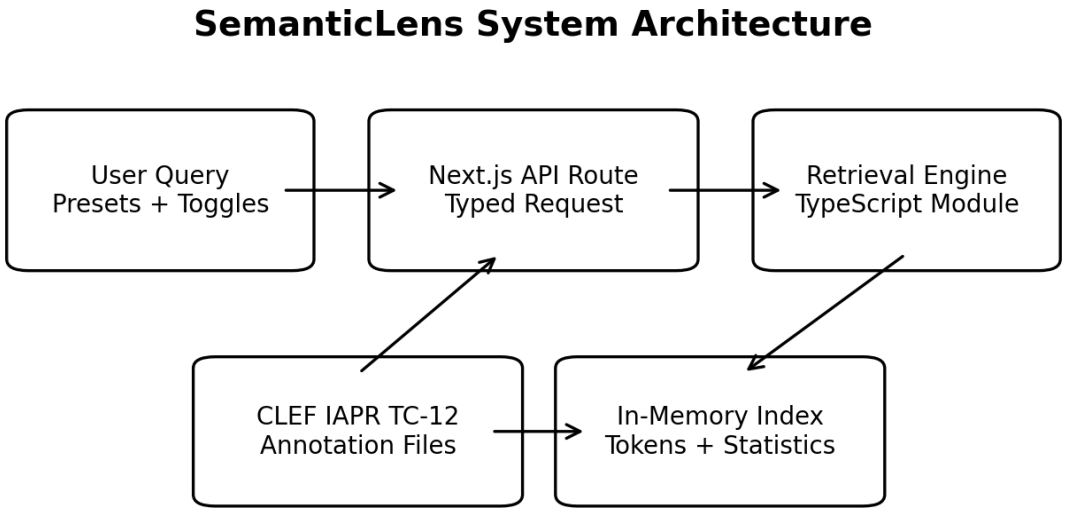


Figure 1. System architecture of SemanticLens, showing the separation between user interaction, API routing, retrieval logic, local annotation files, and the cached index.

1.1 Scholarly Context

The CLEF IAPR TC-12 is a collection of images with captions (titles, locations, dates, descriptions) and text annotations. This makes it useful for retrieval experiments, since it sits between visual search and text search. SemanticLens does not analyze the images themselves. Instead, it searches the text that describes them. Consequently, the quality of retrieval is heavily influenced by how well the system handles the annotation language. Lexical matching is central to classical information retrieval. A document is relevant when the words overlap with query terms and ranking models such as TF-IDF emphasize terms that occur more often in the document than in the wider collection. This strategy remains powerful because it is fast and easy to interpret: the reason a document was returned is usually straightforward. Its main weakness is vocabulary mismatch. A user might search for coast when an annotation says shore, or type car when the annotation says automobile. Lexical resources such as WordNet help address this by organizing words into synonym, hypernym and other conceptual relations. SemanticLens builds on that tradition through a small set of hand-curated, WordNet-style expansion rules. The expansion is intentionally capped. Too much expansion can make a search engine generous in the wrong way and return language-related but not contextually useful material.

1.2 Problem Statement

The primary issue is to enable searchability for a local annotation collection in a manner that is flexible, explainable, and functional. Users require distinct degrees of control. Some searches require precise matching of phrases. Some gain from stemming, synonym expansion, wildcard search, or proximity constraints. No one retrieval interface can take care of these modes without being slow or confusing. Transparency is just as important. A ranked list by itself is insufficient for a data-oriented retrieval site. Users should be able to see why a result appeared, whether the query was changed by spelling correction, whether expanded terms were used, and whether the image shown came from the dataset or from a fallback. These explanations make the system not only usable but also open to inspection. The dataset adds another issue: annotation files can exist even when the corresponding image files are not available. SemanticLens therefore requires a graceful fallback behavior. It still needs to return useful results, but it must not confuse original dataset images with substitute images.

2. System Analysis

This section describes both the capabilities SemanticLens needs to provide and the constraints it operates under. Its intended users are students, researchers and developers who want to explore a collection of annotated images with advanced searches. Many of these users may be unfamiliar with retrieval syntax. The site thus requires both simple entry points and advanced controls. The system is not a basic web search engine. It is designed for a bounded collection with structured annotations, and this narrower focus is an advantage. It also allows the engine to index zones like title, description, notes, location and date, and to expose search controls that are compatible with the structure of the dataset.

2.1 Functional Requirements

The first functional requirement is expressive query input. The system should support plain text, quoted phrases, Boolean expressions, regular expressions, wildcard patterns, and proximity operators. These input types express different intentions. A phrase search supports precision. When the user is unsure of spelling or word form, wildcard search can help. A proximity query expresses the idea that nearby words often indicate a stronger relationship than separated terms. The second requirement is simultaneous use of more than one matching strategy. SemanticLens needs to support normalized token matching, stemming, semantic expansion, and TF-IDF-based scoring. No uniform method can always be satisfactory. Stemming assists with morphology, but does not solve synonymy. Semantic expansion captures conceptual associations, but it may decrease precision. TF-IDF is helpful for ordering distinct matches, but it relies on the terminology in the annotations. User feedback is the third requirement. The interface can also provide spelling corrections, expanded semantic terms, autocomplete options and match explanations. This is consistent with a human-centered retrieval principle: Users can iterate on a system more efficiently when they have the ability to check how it interpreted their query. The fourth requirement is to present results clearly. A title, description, notes, location, date, image preview, and source label must be attached to each retrieved item. These fields turn text retrieval

into a visual browsing experience. The source label is significant since it allows original dataset data to be separated from fallback images.

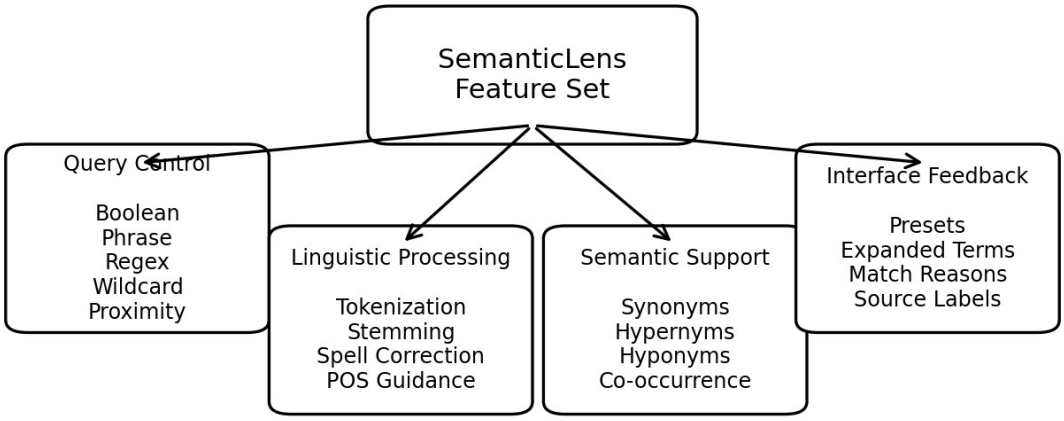


Figure 2. Feature taxonomy of the SemanticLens website, including query controls, linguistic processing, semantic support, and interface feedback features.

2.2 Non-Functional Requirements

Performance is a key non-functional requirement: the website must respond fast enough to feel interactive. Since the dataset is local and bounded, SemanticLens relies on an in-memory index, not a database-based search server. This simplifies setup, but it will not scale to much larger collections. Reliability is also necessary. Invalid regular expressions, query forms not supported, or unexpected annotation content should not crash the application. The search module thus requires defensive parsing and fallback behaviour. Error handling is not an optional extra; it is part of the user experience. TypeScript was chosen for maintainability. Using a typed request and response structure eliminates ambiguity between the front end and the API route. In addition, explicit module boundaries help make code more scalable. A prospective programmer should be able to change semantic expansion rules, the weights of ranking, or the components of an interface without transforming the entire system. Usability is also a key requirement. Advanced retrieval tools easily fail when their power is exposed without guidance. SemanticLens addresses this with presets, grouped toggles, explanatory text and visible result explanations. It does not just search. It also teaches users what the search is doing.

2.3 System Architecture

SemanticLens works in a separation-of-concerns design. The front end deals with input, option selection, feature panels, and result rendering. The API route on the server-side receives the search request and sends that to the retrieval engine. The retrieval engine loads the annotation files, constructs or returns an index, evaluates the query, ranks the results, and provides structured results. The front end is implemented by React and is placed inside a Next.js page. React state will contain query text, selected feature toggles, zone filters, stemmer choice, ranking options and returned results. Through a POST request, the page interfaces with the API route. The search options are sent as a typed payload, which keeps the client and the server consistent. The search engine is implemented as a separate TypeScript module. It consumes annotation files from the local file system and creates an in-memory representation of normalized text, token positions, document frequencies, and co-occurrence counts. The index is built on the first request and then cached, so it does not have to be rebuilt for every search. This is a pragmatic compromise: the system is not a full search server, but it stays transparent, works directly with the annotation files and responds in real time.

3. System Design

During the design phase, we turned the analysis into concrete system structures. SemanticLens deliberately avoids a heavyweight database or distributed architecture; instead, the application uses local annotation files, an in-memory index, a typed API layer and a responsive web interface. The result is accessible, yet still able to demonstrate sophisticated retrieval behavior.

3.1 Data and Conceptual Model

The document model treats each annotation as a retrievable unit. Each document consists of fields such as title, description, notes, location and date.

The index stores normalized forms of these fields with token positions and term statistics. This representation is used for zone filtering, phrase matching, proximity matching and TF-IDF scoring. We can describe a retrieval engine as a collection of cooperating elements. A tokenizer prepares text for matching. A stemmer reduces morphological variation. A query parser finds operators and unique syntax. When enabled, a semantic expansion module adds related terms to the query. A scorer pools evidence of lexical matches, semantic relationships, and TF-IDF weights. Finally, a formatter returns the results together with their explanations.

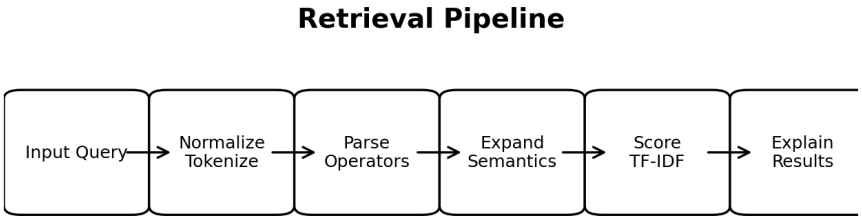


Figure 3. Retrieval pipeline from query input to ranked and explained results.

3.2 Optional Database Design

A persistent database was considered but is not required for the current version. Because the CLEF IAPR TC-12 annotation subset has a scale that can be optimized within memory, a file-based model with an in-memory index is an acceptable methodology. This eliminates the complexity of operations and focuses attention on retrieval logic rather than infrastructure. A future production release might add a relational schema with tables containing documents, fields, terms, postings, semantic relationships and others. Such a structure would allow for incremental updating and ongoing indexing. Another option would be document-based search engines, e.g. Elasticsearch or OpenSearch. Mature-level tokenization, fuzzy search, synonym filters and ranking are supported by these platforms, but they make setup more complex and offer less insight into how the retrieval logic works.

3.3 Interface Design

The interface is designed as a dashboard rather than a single search box. This choice reflects the educational purpose of the project. Users can search as normal, but they can also see grouped controls on lexical features, semantic features, ranking options, and display behavior. Rather than concealing the retrieval process, the interface invites experimentation. Preset queries reduce the barrier of entry for users who are not familiar with the syntax that is available. They also show what the system can do. Presets for landmarks, nature scenes, operator examples and semantic probes keep the search engine approachable without removing advanced functionality. Result cards are where metadata meets visual evidence. Each card shows the annotation title, description, notes, location, date, and preview of the image. It also identifies if the image shown is from the dataset or an external fallback. This design balances usability and research integrity, as the user will be able to separate retrieved metadata from substituted visual content.

4. Implementation

Implementation takes the design into a functioning application. SemanticLens uses Next.js for routing and API support, React for interactive interface behavior, and TypeScript for type safety. This combination suits a project whose interactive client must communicate reliably with server-side search logic. Supporting libraries are used only selectively. Framer Motion supports interface transitions and result rendering. Lucide React offers icons to differentiate feature categories. Utility-based styling keeps the layout consistent without a large custom stylesheet. These choices keep the project lean while still polished.

4.1 Retrieval Techniques

The initial layer of retrieval is tokenization and normalization. Raw annotation text is split into lowercase tokens for each field, and punctuation is handled consistently across all fields. This is a simple but crucial step. Later methods, such as stemming, phrase matching and TF-IDF scoring, are unreliable without consistent preprocessing. The system offers simplified, rule-based versions of the Porter, Snowball and Lancaster stemmers, as well as a manual stemmer. Stemming reduces morphological variation by mapping related word forms to a common root. This helps when the query and the document use different forms of the same word. It also has limits. Stemming can merge words that should stay separate. SemanticLens turns this limitation into a teaching opportunity by letting users choose the stemmer and compare the results. Boolean matching supports operators such as A AND B, A OR B and A AND NOT B, which let users control precision and exclude unwanted terms. Exact phrase matching handles quoted strings, which is useful when word order is important. Proximity matching, for example palm /2 tree, finds terms that appear within a specified distance of each other. This captures relationships stronger than simple co-occurrence anywhere in the annotation. Wildcard matching allows for leading, trailing, and internal wildcard patterns. The system transforms wildcard expressions into regular expressions that can match many token forms. Regular expression mode provides more control to advanced users, but input must be validated carefully so that malformed patterns do not break the search. Semantic expansion uses a compact, hand-curated dictionary of WordNet-, MeSH- and YAGO-style relations. The system then adds synonyms, hypernyms, hyponyms, and related terms to the query profile. This addresses vocabulary mismatch while keeping the expansion selective. While a small curated expansion set is less diverse than a complete ontology, it can minimize unnecessary augmentation. Context-aware expansion takes co-occurrence statistics available from the dataset. If a query term has no direct semantic relations in the dictionary, commonly co-occurring terms provide a local form of background knowledge. This works well for collection-specific language, but it has a pitfall: co-occurrence doesn't always mean semantic similarity. As a result, the system considers context expansion as an auxiliary signal, and not a substitute for lexical evidence.

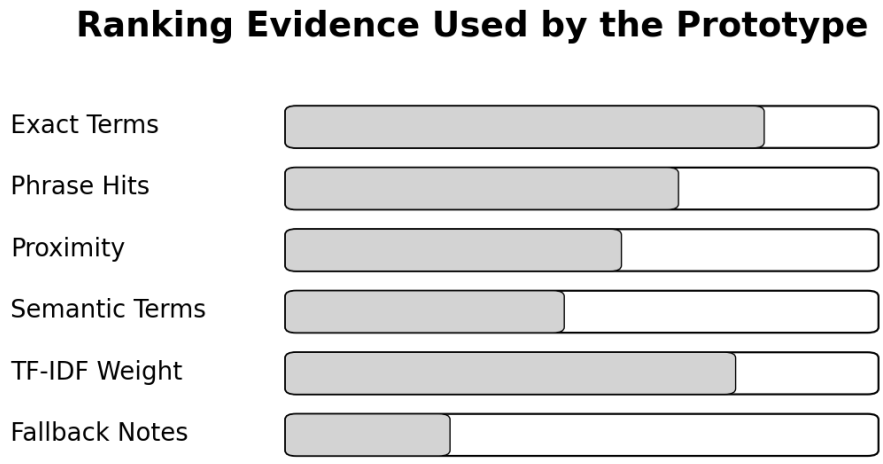


Figure 4. Interpretable ranking evidence combined by the prototype.

4.2 Website Features Added for User Interaction

The website also includes many features that make it more than a simple search box. Users can select stemmers, enable or disable semantic expansion, switch ranking modes, limit search to specific document zones, try Boolean and proximity operators, use wildcard and regular expression matching and check why a result was provided. These controls are not decorative. They enable users to directly compare retrieval activity. With autocomplete and preset examples, users can start searching without memorizing syntax. When the user enters a term not in the dataset vocabulary, spelling correction comes into play. Expanded-term displays show how a semantic module altered the query. Match reasons explain the part of the engine that contributed to a result. Together, these features keep SemanticLens from being a black box and turn it into a small laboratory for retrieval behavior. The fallback image functionality is important too. When a local image is not available, the website shows a placeholder or an external image, but it clearly labels the source. It is a small, interface-based decision that carries big effects. It safeguards the user from mistaking retrieved metadata for true visual evidence.

4.3 Feature Summary

Feature Area	**Website Feature**	**Purpose**
Query syntax	Boolean, phrase, proximity,	Supports different levels of

	wildcard, and regex search	precision and exploration
Linguistic processing	Tokenization, normalization, stemming, and spell correction	Reduces surface-level variation and user input errors
Semantic retrieval	Synonyms, hypernyms, hyponyms, related terms, and co-occurrence	Addresses vocabulary mismatch without relying only on exact words
Ranking	TF-IDF and rule-based scoring evidence	Prioritizes distinctive and interpretable matches
Interface	Presets, toggles, result cards, match explanations, and source labels	Makes advanced retrieval understandable and transparent

Table 1. Summary of major website features and their retrieval purpose.

5. Critical Evaluation and Future Work

SemanticLens works best as an explainable retrieval prototype. Its greatest advantage is not just that it returns results, but that it reveals how they were produced. Users can view expanded terms, spelling recommendations, feature states and source labels. In the scholarly context, this transparency becomes critical, for the retrieval process is as important as the outcome. The work further highlights the utility of hybrid retrieval. Lexical matching adds precision and interpretability. Semantic expansion deals with vocabulary mismatch. TF-IDF ranking lets the system prioritize distinctive matches in a principled way. Interface presets and explanations render these methods easily accessible to non-specialists. These elements combined constitute a system that is both technically sound and pedagogically valuable. At the same time, the project suffers from several limitations. The semantic expansion module is highly curated and compact, which reduces coverage. Expansion can fail to help a query that is not supported by semantic mappings. The co-occurrence module is highly dependent on the collection and might easily mistake repeated association as relevant similarity. The ranking model is interpretable, but it is simpler than modern dense retrieval techniques. Finally, the file-based architecture is suitable for the project implementation scale, but would necessitate redesign for extremely large datasets or concurrent production use. Future work might branch off into a number of directions. A persistent database or search index would allow for larger datasets and incremental updates. Dense embedding retrieval could provide semantic similarity beyond explicit lexical pairs in practice. Connecting to full lexical resources such as WordNet, domain-specific ontologies or multilingual lexical databases could further improve the expansion. Evaluation dashboards would compare precision, recall and ranking quality by search setting. Collaborative annotation features might turn the website from a retrieval interface into a data enhancement tool.

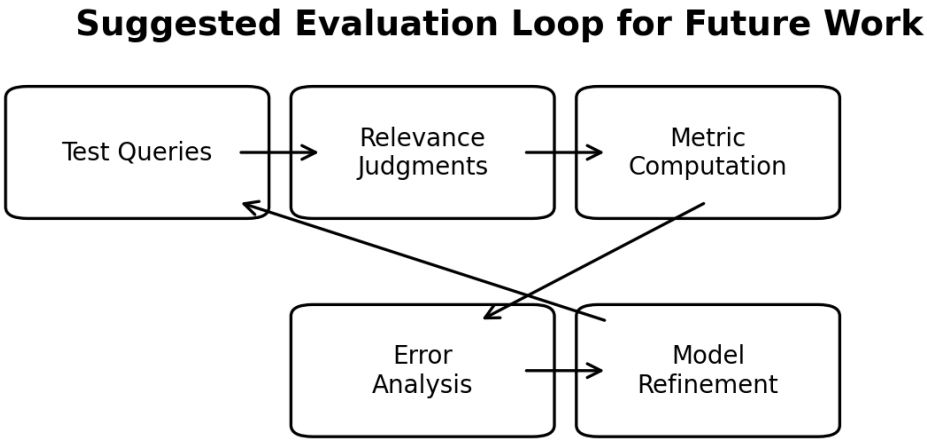


Figure 5. Proposed evaluation workflow for developing SemanticLens from a prototype into a more rigorous research platform.

The most significant future enhancement would be a systematic evaluation. The current version demonstrates its potential only through examples. A stronger research version would define a set of test queries, relevance judgments and metrics such as precision at k, recall, mean average precision and normalized discounted cumulative gain. This would make it possible to compare lexical search, semantic expansion, context expansion and TF-IDF ranking in a controlled experiment.

6. Conclusion

SemanticLens demonstrates how an annotated image dataset can be converted into an interactive retrieval website using NLP preprocessing, semantic enrichment, ranked retrieval and smart interface design. Its main strength is that it makes search behavior visible. It doesn’t require users to trust an opaque model. It provides them with the tools they need to inspect and tweak the retrieval process. The project also teaches us a broader lesson in applied NLP. Effective retrieval is not just about adding more techniques. It calls for judgment on when to focus on precision, when to widen recall, how to explain results, and how to maintain usability. SemanticLens is designed to solve these problems by being lightweight and structured. As it stands, it is a solid prototype. With thorough evaluation and an improved indexing layer, it could become a more rigorous platform for semantic search over annotated multimedia collections.

References

Azad, Hiteshwar Kumar, and Akshay Deepak. “A New Approach for Query Expansion Using Wikipedia and WordNet.” *arXiv*, 2019, arxiv.org/abs/1901.10197.

Barz, Björn, and Joachim Denzler. “Content-Based Image Retrieval and the Semantic Gap in the Deep Learning Era.” *arXiv*, 2020, arxiv.org/abs/2011.06490.

CLEF/ImageCLEF. “IAPR TC-12 Benchmark.” *ImageCLEF*, www.imageclef.org/photodata.

Escalante, Hugo Jair, et al. “The Segmented and Annotated IAPR TC-12 Benchmark.” *Computer Vision and Image Understanding*, vol. 114, no. 4, 2010, pp. 419–428.

Farivar, Kayla. “Semantic Search for Information Retrieval.” *arXiv*, 2025, arxiv.org/abs/2508.17694.

Grubinger, Michael, Paul Clough, Henning Müller, and Thomas Deselaers. “The IAPR TC-12 Benchmark: A New Evaluation Resource for Visual Information Systems.” *Proceedings of the International Conference on Language Resources and Evaluation (LREC)*, 2006, pp. 13–23.

Li, Wei, et al. “A Cross-Modal Hash Retrieval Method with Fused Triples.” *Applied Sciences*, vol. 13, no. 18, 2023, article 10524.

Manning, Christopher D., Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge UP, 2008.

Ngo, Vuong M., Tru H. Cao, and Tuan M. V. Le. “WordNet-Based Information Retrieval Using Common Hypernyms and Combined Features.” *arXiv*, 2018, arxiv.org/abs/1807.05574.

Porter, Martin F. “An Algorithm for Suffix Stripping.” *Program*, vol. 14, no. 3, 1980, pp. 130–137.

Vijayarajan, V., et al. “A Generic Framework for Ontology-Based Information Retrieval and Image Retrieval.” *EURASIP Journal on Information Security*, vol. 2016, no. 1, 2016.

Zhang, Le, et al. “Exploring the Best Practices of Query Expansion with Large Language Models.” *Findings of the Association for Computational Linguistics: EMNLP 2024*, Association for Computational Linguistics, 2024, pp. 1872–1883.